

# CMSA-Net for Crop Weed Segmentation

## Cross-Modal Spatial Attention Network — Complete Source Code

---

```
"""
=====
CMSA-NET: CROSS-MODAL SPATIAL ATTENTION NETWORK
Complete Implementation for WeedsGalore Dataset (3-Class Version)
=====

Novel Contributions:
1. [KEY] Cross-Modal Spatial Attention - Pixel-wise fusion (vs global weights)
2. [KEY] Hierarchical Multi-Scale Fusion - 4 decoder levels (vs 1 bottleneck)
3. [KEY] Boundary-Aware Refinement - Explicit boundary supervision

Target: 82-84% mIoU (3-class)

=====
"""

# =====
# SECTION 1: SETUP & IMPORTS
# =====

print("="*80)
print("SECTION 1: SETUP & IMPORTS")
print("="*80)

import subprocess
import sys

# Install required packages
packages = ['albumentations']
for package in packages:
    try:
        __import__(package)
    except ImportError:
        print(f"Installing {package}...")
        subprocess.check_call([sys.executable, "-m", "pip", "install", "-q", package])

# Core imports
import os
import numpy as np
import pandas as pd
import cv2
from PIL import Image
import matplotlib.pyplot as plt
from tqdm.auto import tqdm
import glob
import json
import random
import warnings
from collections import defaultdict
warnings.filterwarnings('ignore')

# Deep learning
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader
import torch.optim as optim
from torch.cuda.amp import autocast, GradScaler

# Albumentations
import albumentations as A

print("\nAll packages imported!")
print(f"PyTorch version: {torch.__version__}")
```

```

print(f"CUDA available: {torch.cuda.is_available()}")

if torch.cuda.is_available():
    print(f"GPU: {torch.cuda.get_device_name(0)}")
    print(f"Memory: {torch.cuda.get_device_properties(0).total_memory / 1e9:.2f} GB")

# Set seeds
def set_seed(seed=42):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
    torch.backends.cudnn.deterministic = True

set_seed(42)
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"\nUsing device: {device}")

print("\n" + "="*80)
print("SECTION 1 COMPLETE")
print("="*80 + "\n")

# =====
# SECTION 2: CONFIGURATION
# =====
print("="*80)
print("SECTION 2: CONFIGURATION")
print("="*80)

class Config:
    """Configuration for CMSA-Net training"""

    # =====
    # DATASET PATHS - UPDATE THIS!
    # =====
    BASE_DIR = "/content/drive/MyDrive/weedsgalore-dataset" # UPDATE THIS!
    SPLITS_DIR = os.path.join(BASE_DIR, "splits")

    # =====
    # SAVE LOCATIONS
    # =====
    MODEL_NAME = "CMSANet_WeedsGalore_3Class"
    MODEL_SAVE_PATH = f'/content/drive/MyDrive/{MODEL_NAME}_best.pth'
    CHECKPOINT_DIR = f'/content/drive/MyDrive/{MODEL_NAME}_checkpoints/'
    LOG_PATH = f'/content/drive/MyDrive/{MODEL_NAME}_training.csv'
    VIS_DIR = f'/content/drive/MyDrive/{MODEL_NAME}_visualizations/'

    os.makedirs(CHECKPOINT_DIR, exist_ok=True)
    os.makedirs(VIS_DIR, exist_ok=True)

    # =====
    # TASK CONFIGURATION - 3 CLASSES
    # =====
    NUM_CLASSES = 3 # background, crop, weed
    CLASS_NAMES = [
        'background', # Class 0
        'crop', # Class 1 (maize)
        'weed' # Class 2 (all weed types)
    ]
    CLASS_COLORS = [
        [0, 0, 0], # background - black
        [0, 255, 0], # crop - green
        [255, 0, 0] # weed - red
    ]

    # Model architecture
    IMG_SIZE = 600
    IN_CHANNELS = 5

```

```

RGB_FILTERS = [32, 64, 128, 256]
NIR_FILTERS = [16, 32, 64, 128]
RE_FILTERS = [16, 32, 64, 128]
DECODER_DIMS = [256, 128, 64, 32]

# Training hyperparameters
BATCH_SIZE = 4
GRADIENT_ACCUMULATION = 2
LEARNING_RATE = 1e-3
MIN_LR = 1e-6
NUM_EPOCHS = 100
WEIGHT_DECAY = 0.01

# Scheduler
USE_REDUCE_ON_PLATEAU = True
PLATEAU_PATIENCE = 7
PLATEAU_FACTOR = 0.5

# Loss weights
SEG_WEIGHT = 0.7
BOUNDARY_WEIGHT = 0.3
FOCAL_GAMMA = 2.0
FOCAL_ALPHA = 0.25

# Augmentation
AUG_FLIP_PROB = 0.5
AUG_ROTATE_PROB = 0.5

# Training control
EARLY_STOPPING_PATIENCE = 20
SAVE_FREQUENCY = 10
USE_MIXED_PRECISION = True
GRADIENT_CLIP_NORM = 1.0

config = Config()

print(f"\nDataset: {config.BASE_DIR}")
print(f"Model will be saved to: {config.MODEL_SAVE_PATH}")
print(f"\nTarget: 82-84% mIoU (3-class)")
print(f"Classes: {config.CLASS_NAMES}")

# =====
# SECTION 3: DATASET STRUCTURE VALIDATION
# =====
print("="*80)
print("SECTION 3: DATASET STRUCTURE VALIDATION")
print("="*80)

print("\nValidating WeedsGalore dataset structure...")

# Check base directory
if not os.path.exists(config.BASE_DIR):
    print(f"ERROR: Dataset not found: {config.BASE_DIR}")
    print("Please update config.BASE_DIR!")
    raise FileNotFoundError(f"Dataset not found: {config.BASE_DIR}")
else:
    print(f"Base directory exists: {config.BASE_DIR}")

# Check date folders
print("\nChecking date folders...")
date_folders = sorted([d for d in os.listdir(config.BASE_DIR)
                       if os.path.isdir(os.path.join(config.BASE_DIR, d))
                       and d.startswith('2023')])

if not date_folders:
    print("ERROR: No date folders found!")
else:
    print(f"Found {len(date_folders)} date folders:")
    for folder in date_folders:

```

```

folder_path = os.path.join(config.BASE_DIR, folder)
images_dir = os.path.join(folder_path, 'images')
semantics_dir = os.path.join(folder_path, 'semantics')

if os.path.exists(images_dir) and os.path.exists(semantics_dir):
    image_files = [f for f in os.listdir(images_dir) if f.endswith('.png')]
    mask_files = [f for f in os.listdir(semantics_dir) if f.endswith('.png')]
    unique_tiles = len(set([f.rsplit('_', 1)[0] for f in image_files]))
    print(f"    - {folder}: {unique_tiles} tiles, {len(mask_files)} masks")

# Check split files
print("\nChecking split files...")
split_files = ['train.txt', 'val.txt', 'test.txt']
split_counts = {}

for split_file in split_files:
    split_path = os.path.join(config.SPLITS_DIR, split_file)
    if os.path.exists(split_path):
        with open(split_path, 'r') as f:
            lines = [line.strip() for line in f if line.strip()]
            split_counts[split_file.replace('.txt', '')] = len(lines)
            print(f"FOUND {split_file}: {len(lines)} tiles")
    else:
        print(f"NOT FOUND: {split_file}")

print(f"\nTotal: {sum(split_counts.values())} tiles")

print("\n" + "="*80)
print("SECTION 3 COMPLETE")
print("="*80 + "\n")

# =====
# SECTION 4: DATASET CLASS
# =====
print("="*80)
print("SECTION 4: DATASET CLASS")
print("="*80)

class WeedsGaloreDataset(Dataset):
    """
    WeedsGalore Dataset Loader (3-Class Version)

    Remaps original 6 classes to 3:
    - 0: background -> 0
    - 1: maize (crop) -> 1
    - 2-5: all weeds -> 2
    """

    def __init__(self, base_dir, splits_dir, split='train',
                 transform=None, img_size=600):
        self.base_dir = base_dir
        self.splits_dir = splits_dir
        self.split = split
        self.transform = transform
        self.img_size = img_size

        self.samples = self._load_split()
        print(f"    Loaded {len(self.samples)} tiles for {self.split}")

    def _load_split(self):
        """Load tile IDs from split file"""
        split_file = os.path.join(self.splits_dir, f'{self.split}.txt')

        with open(split_file, 'r') as f:
            tile_ids = [line.strip() for line in f if line.strip()]

        samples = []
        missing = 0

```

```

for tile_id in tile_ids:
    # Extract date folder (format: 2023-05-25_0039)
    date_folder = tile_id[:10]

    images_dir = os.path.join(self.base_dir, date_folder, 'images')
    semantics_dir = os.path.join(self.base_dir, date_folder, 'semantics')

    # Build paths for all 5 bands
    band_paths = {
        'R': os.path.join(images_dir, f"{tile_id}_R.png"),
        'G': os.path.join(images_dir, f"{tile_id}_G.png"),
        'B': os.path.join(images_dir, f"{tile_id}_B.png"),
        'NIR': os.path.join(images_dir, f"{tile_id}_NIR.png"),
        'RE': os.path.join(images_dir, f"{tile_id}_RE.png"),
    }
    mask_path = os.path.join(semantics_dir, f"{tile_id}.png")

    # Verify all files exist
    all_exist = all(os.path.exists(p) for p in band_paths.values())
    all_exist = all_exist and os.path.exists(mask_path)

    if all_exist:
        samples.append({
            'tile_id': tile_id,
            'band_paths': band_paths,
            'mask_path': mask_path
        })
    else:
        missing += 1

if missing > 0:
    print(f"    WARNING: {missing} tiles missing files (skipped)")

return samples

def _normalize_band(self, band_array):
    """Normalize band to [0, 1] using percentile clipping"""
    band_array = band_array.astype(np.float32)
    p2, p98 = np.percentile(band_array, (2, 98))
    band_array = np.clip(band_array, p2, p98)
    band_array = (band_array - p2) / (p98 - p2 + 1e-8)
    return np.clip(band_array, 0, 1)

def _remap_mask(self, mask):
    """
    Remap 6-class mask to 3-class:
    0 -> 0 (background)
    1 -> 1 (crop/maize)
    2,3,4,5 -> 2 (all weeds)
    """
    mask_3class = mask.copy()
    # Keep 0 and 1 as is
    # Remap all weed classes (2,3,4,5) to class 2
    mask_3class[mask >= 2] = 2
    return mask_3class

def __len__(self):
    return len(self.samples)

def __getitem__(self, idx):
    sample = self.samples[idx]

    # Load all 5 bands
    bands = []
    for band_name in ['R', 'G', 'B', 'NIR', 'RE']:
        band_path = sample['band_paths'][band_name]
        band = np.array(Image.open(band_path), dtype=np.float32)
        band = self._normalize_band(band)
        bands.append(band)

```

```

# Stack: [R, G, B, NIR, RE]
image = np.stack(bands, axis=-1) # (H, W, 5)

# Load mask
mask = np.array(Image.open(sample['mask_path']), dtype=np.int64)
mask = self._remap_mask(mask) # Convert to 3-class

# Resize if needed
if image.shape[0] != self.img_size or image.shape[1] != self.img_size:
    image = cv2.resize(image, (self.img_size, self.img_size),
                       interpolation=cv2.INTER_LINEAR)
    mask = cv2.resize(mask, (self.img_size, self.img_size),
                      interpolation=cv2.INTER_NEAREST)

# Apply augmentations
if self.transform:
    augmented = self.transform(image=image, mask=mask)
    image = augmented['image']
    mask = augmented['mask']

# To tensors
if isinstance(image, np.ndarray):
    image = torch.from_numpy(image).permute(2, 0, 1).float()
    mask = torch.from_numpy(np.array(mask)).long()

    return image, mask, sample['tile_id']

print("Dataset class defined (with 6->3 class remapping)!")

print("\n" + "="*80)
print("SECTION 4 COMPLETE")
print("="*80 + "\n")

# =====
# SECTION 5: TEST DATASET LOADING
# =====
print("="*80)
print("SECTION 5: TEST DATASET LOADING")
print("="*80)

print("\nCreating dataset instances...")

train_dataset = WeedsGaloreDataset(
    config.BASE_DIR, config.SPLITS_DIR, 'train',
    transform=None, img_size=config.IMG_SIZE
)

val_dataset = WeedsGaloreDataset(
    config.BASE_DIR, config.SPLITS_DIR, 'val',
    transform=None, img_size=config.IMG_SIZE
)

print(f"\nDatasets created:")
print(f"  Train: {len(train_dataset)} tiles")
print(f"  Val:   {len(val_dataset)} tiles")

# Test loading
print("\nTesting sample loading...")
sample_img, sample_mask, sample_id = train_dataset[0]

print(f"\nSample loaded:")
print(f"  ID: {sample_id}")
print(f"  Image shape: {sample_img.shape}")
print(f"  Image range: [{sample_img.min():.3f}, {sample_img.max():.3f}]")
print(f"  Mask shape: {sample_mask.shape}")
print(f"  Mask classes: {torch.unique(sample_mask).tolist()}")
print(f"  Expected: [0, 1, 2] for [background, crop, weed]")
print("\n" + "="*80)

```

```

print("SECTION 5 COMPLETE")
print("="*80 + "\n")

# =====
# SECTION 6: DATA AUGMENTATION
# =====
print("="*80)
print("SECTION 6: DATA AUGMENTATION")
print("="*80)

print("\nCreating augmentation pipeline...")

train_transform = A.Compose([
    A.HorizontalFlip(p=config.AUG_FLIP_PROB),
    A.VerticalFlip(p=config.AUG_FLIP_PROB),
    A.RandomRotate90(p=config.AUG_ROTATE_PROB),
])

print("Augmentation defined!")
print(f"    H/V flip: {config.AUG_FLIP_PROB}")
print(f"    Rotate 90 degrees: {config.AUG_ROTATE_PROB}")

print("\n" + "="*80)
print("SECTION 6 COMPLETE")
print("="*80 + "\n")

# =====
# SECTION 7: CMSA-NET ARCHITECTURE
# =====
print("="*80)
print("SECTION 7: CMSA-NET ARCHITECTURE (NOVEL)")
print("="*80)

print("\nBuilding CMSA-Net with 3 novel components:")
print("    1. [KEY] Cross-Modal Spatial Attention")
print("    2. [KEY] Hierarchical Multi-Scale Fusion")
print("    3. [KEY] Boundary-Aware Refinement")

# =====
# NOVEL MODULE 1: Cross-Modal Spatial Attention
# =====

class CrossModalSpatialAttention(nn.Module):
    """
    [KEY] NOVELTY #1: Spatial attention for multimodal fusion

    Paper approach: fusion = 0.4*RGB + 0.3*NIR + 0.3*RE (global weights)
    Our approach: fusion = A_RGB*RGB + A_NIR*NIR + A_RE*RE (per-pixel weights)
    """

    def __init__(self, rgb_dim, nir_dim, re_dim, out_dim):
        super().__init__()

        self.rgb_proj = nn.Conv2d(rgb_dim, out_dim, 1, bias=False)
        self.nir_proj = nn.Conv2d(nir_dim, out_dim, 1, bias=False)
        self.re_proj = nn.Conv2d(re_dim, out_dim, 1, bias=False)

        # [KEY] Spatial attention generator
        self.spatial_attention = nn.Sequential(
            nn.Conv2d(out_dim * 3, out_dim, 3, padding=1, bias=False),
            nn.BatchNorm2d(out_dim),
            nn.ReLU(inplace=True),
            nn.Conv2d(out_dim, out_dim // 4, 1, bias=False),
            nn.BatchNorm2d(out_dim // 4),
            nn.ReLU(inplace=True),
            nn.Conv2d(out_dim // 4, 3, 1),

```

```

        nn.Softmax(dim=1)
    )

    # Refinement (depthwise separable)
    self.refine = nn.Sequential(
        nn.Conv2d(out_dim, out_dim, 3, padding=1, groups=out_dim, bias=False),
        nn.BatchNorm2d(out_dim),
        nn.ReLU(inplace=True),
        nn.Conv2d(out_dim, out_dim, 1, bias=False),
        nn.BatchNorm2d(out_dim),
        nn.ReLU(inplace=True)
    )

def forward(self, rgb_feat, nir_feat, re_feat):
    # Project
    rgb = self.rgb_proj(rgb_feat)
    nir = self.nir_proj(nir_feat)
    re = self.re_proj(re_feat)

    # Concatenate
    concat = torch.cat([rgb, nir, re], dim=1)

    # [KEY] Generate spatial attention
    attn_maps = self.spatial_attention(concat) # [B, 3, H, W]

    # Split
    w_rgb = attn_maps[:, 0:1, :, :]
    w_nir = attn_maps[:, 1:2, :, :]
    w_re = attn_maps[:, 2:3, :, :]

    # [KEY] Pixel-wise fusion
    fused = w_rgb * rgb + w_nir * nir + w_re * re
    fused = self.refine(fused)

    return fused, attn_maps

# =====
# NOVEL MODULE 2: Hierarchical Multi-Scale Fusion
# =====

class HierarchicalFusionDecoder(nn.Module):
    """
    [KEY] NOVELTY #2: Multi-scale fusion in decoder

    Paper: Fuses at bottleneck only (1 scale)
    Ours: Fuses at EVERY decoder level (4 scales)
    """

    def __init__(self, rgb_filters, nir_filters, re_filters, decoder_dims):
        super().__init__()

        self.num_levels = len(decoder_dims)

        # Fusion at each scale
        self.fusion_modules = nn.ModuleList()
        for i in range(self.num_levels):
            idx = -(i+1)
            fusion = CrossModalSpatialAttention(
                rgb_dim=rgb_filters[idx],
                nir_dim=nir_filters[idx],
                re_dim=re_filters[idx],
                out_dim=decoder_dims[i]
            )
            self.fusion_modules.append(fusion)

        # Upsampling
        self.upsample_modules = nn.ModuleList()
        for i in range(self.num_levels - 1):

```



```

        upsample = nn.Sequential(
            nn.ConvTranspose2d(decoder_dims[i], decoder_dims[i+1],
                               4, stride=2, padding=1, bias=False),
            nn.BatchNorm2d(decoder_dims[i+1]),
            nn.ReLU(inplace=True)
        )
        self.upsample_modules.append(upsample)

def forward(self, rgb_feats, nir_feats, re_feats):
    decoded_feats = []
    attn_maps = []

    for i in range(self.num_levels):
        rgb_feat = rgb_feats[-(i+1)]
        nir_feat = nir_feats[-(i+1)]
        re_feat = re_feats[-(i+1)]

        # Fuse at this scale
        fused, attn = self.fusion_modules[i](rgb_feat, nir_feat, re_feat)

        # Add skip from previous level
        if i > 0:
            upsampled = self.upsample_modules[i-1](decoded_feats[-1])
            if upsampled.shape[2:] != fused.shape[2:]:
                upsampled = F.interpolate(upsampled, size=fused.shape[2:],
                                           mode='bilinear', align_corners=False)
            fused = fused + upsampled

        decoded_feats.append(fused)
        attn_maps.append(attn)

    return decoded_feats, attn_maps

# =====
# NOVEL MODULE 3: Boundary-Aware Refinement
# =====

class BoundaryRefinementModule(nn.Module):
    """
    [KEY] NOVELTY #3: Explicit boundary refinement

    Detects and refines boundaries for sharp segmentation
    """

    def __init__(self, in_channels, num_classes):
        super().__init__()

        # Boundary detector
        self.boundary_detector = nn.Sequential(
            nn.Conv2d(in_channels, 64, 3, padding=1, bias=False),
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True),
            nn.Conv2d(64, 32, 3, padding=1, bias=False),
            nn.BatchNorm2d(32),
            nn.ReLU(inplace=True),
            nn.Conv2d(32, 1, 1)
        )

        # Boundary-guided refinement
        self.refiner = nn.Sequential(
            nn.Conv2d(in_channels + 1, in_channels, 3, padding=1, bias=False),
            nn.BatchNorm2d(in_channels),
            nn.ReLU(inplace=True),
            nn.Conv2d(in_channels, in_channels, 3, padding=1, bias=False),
            nn.BatchNorm2d(in_channels),
            nn.ReLU(inplace=True)
        )

        self.seg_head = nn.Conv2d(in_channels, num_classes, 1)

```

```

def forward(self, features):
    boundary_logits = self.boundary_detector(features)
    boundary_prob = torch.sigmoid(boundary_logits)

    refined = torch.cat([features, boundary_prob], dim=1)
    refined = self.refiner(refined)

    seg_logits = self.seg_head(refined)

    return seg_logits, boundary_logits

# =====
# ENCODER
# =====

class LightweightEncoder(nn.Module):
    """Encoder that returns all intermediate features"""

    def __init__(self, in_channels, filters):
        super().__init__()

        self.stages = nn.ModuleList()
        prev_ch = in_channels

        for out_ch in filters:
            stage = nn.Sequential(
                nn.Conv2d(prev_ch, out_ch, 3, stride=2, padding=1, bias=False),
                nn.BatchNorm2d(out_ch),
                nn.ReLU(inplace=True)
            )
            self.stages.append(stage)
            prev_ch = out_ch

    def forward(self, x):
        features = []
        for stage in self.stages:
            x = stage(x)
            features.append(x)
        return features

# =====
# MAIN MODEL: CMSA-NET
# =====

class CMSANet(nn.Module):
    """
    Cross-Modal Spatial Attention Network

    Novel: Pixel-wise fusion + Multi-scale fusion + Boundary refinement
    Expected: 82-84% mIoU
    """

    def __init__(self, num_classes=3):
        super().__init__()

        self.rgb_filters = config.RGB_FILTERS
        self.nir_filters = config.NIR_FILTERS
        self.re_filters = config.RE_FILTERS
        self.decoder_dims = config.DECODER_DIMS

        # Encoders
        self.rgb_encoder = LightweightEncoder(3, self.rgb_filters)
        self.nir_encoder = LightweightEncoder(1, self.nir_filters)
        self.re_encoder = LightweightEncoder(1, self.re_filters)

        # [KEY] Hierarchical fusion
        self.hierarchical_fusion = HierarchicalFusionDecoder(

```

```

        self.rgb_filters, self.nir_filters, self.re_filters, self.decoder_dims
    )

    # [KEY] Boundary refinement
    self.boundary_refinement = BoundaryRefinementModule(
        self.decoder_dims[-1], num_classes
    )

def forward(self, x):
    input_size = x.shape[2:]

    # Split modalities
    rgb = x[:, :3]
    nir = x[:, 3:4]
    re = x[:, 4:5]

    # Encode
    rgb_feats = self.rgb_encoder(rgb)
    nir_feats = self.nir_encoder(nir)
    re_feats = self.re_encoder(re)

    # [KEY] Hierarchical fusion
    fused_feats, attn_maps = self.hierarchical_fusion(rgb_feats, nir_feats, re_feats)

    # [KEY] Boundary refinement
    seg_logits, boundary_logits = self.boundary_refinement(fused_feats[-1])

    # Resize to input
    if seg_logits.shape[2:] != input_size:
        seg_logits = F.interpolate(seg_logits, size=input_size,
                                   mode='bilinear', align_corners=False)
        boundary_logits = F.interpolate(boundary_logits, size=input_size,
                                       mode='bilinear', align_corners=False)

    return seg_logits, boundary_logits, attn_maps

# Create model
model = CMSANet(num_classes=config.NUM_CLASSES).to(device)

total_params = sum(p.numel() for p in model.parameters() if p.requires_grad)

print(f"\nCMSA-NET CREATED:")
print(f"  Parameters: {total_params/1e6:.2f}M")
print(f"  Classes: {config.NUM_CLASSES}")
print(f"  Target: 82-84% mIoU")

# Test
model.eval()
with torch.no_grad():
    test_input = torch.randn(2, 5, 600, 600).to(device)
    test_seg, test_boundary, test_attn = model(test_input)
    print(f"\nForward pass successful:")
    print(f"  Segmentation: {test_seg.shape}")
    print(f"  Boundary: {test_boundary.shape}")
    print(f"  Attention: {len(test_attn)} scales")

del test_input, test_seg, test_boundary, test_attn
torch.cuda.empty_cache()

print("\n" + "="*80)
print("SECTION 7 COMPLETE")
print("="*80 + "\n")

# =====
# SECTION 8: LOSS FUNCTIONS
# =====
print("="*80)

```

```
print("SECTION 8: MULTI-TASK LOSS")
print("="*80)
```

```
class FocalLoss(nn.Module):
    """Focal Loss for class imbalance"""
    def __init__(self, alpha=0.25, gamma=2.0):
        super().__init__()
        self.alpha = alpha
        self.gamma = gamma

    def forward(self, inputs, targets):
        ce_loss = F.cross_entropy(inputs, targets, reduction='none')
        pt = torch.exp(-ce_loss)
        return (self.alpha * ((1 - pt) ** self.gamma) * ce_loss).mean()
```

```
class CMSALoss(nn.Module):
    """Multi-task loss: Segmentation + Boundary"""

    def __init__(self, seg_weight=0.7, boundary_weight=0.3):
        super().__init__()

        self.seg_weight = seg_weight
        self.boundary_weight = boundary_weight

        self.ce_loss = nn.CrossEntropyLoss()
        self.focal_loss = FocalLoss()
        self.boundary_loss = nn.BCEWithLogitsLoss()

    def extract_boundaries(self, mask):
        """Extract boundaries using Sobel"""
        mask_float = mask.float()

        sobel_x = torch.tensor([[[-1, 0, 1], [-2, 0, 2], [-1, 0, 1]],
                                [dtype=torch.float32, device=mask.device).view(1, 1, 3, 3)
        sobel_y = torch.tensor([[[-1, -2, -1], [0, 0, 0], [1, 2, 1]],
                                [dtype=torch.float32, device=mask.device).view(1, 1, 3, 3)

        grad_x = F.conv2d(mask_float.unsqueeze(1), sobel_x, padding=1)
        grad_y = F.conv2d(mask_float.unsqueeze(1), sobel_y, padding=1)

        boundaries = torch.sqrt(grad_x**2 + grad_y**2)
        return (boundaries > 0.1).float().squeeze(1)

    def forward(self, seg_logits, boundary_logits, targets):
        # Segmentation
        ce = self.ce_loss(seg_logits, targets)
        focal = self.focal_loss(seg_logits, targets)
        seg_loss = 0.5 * ce + 0.5 * focal

        # Boundary
        target_boundaries = self.extract_boundaries(targets)
        boundary_loss = self.boundary_loss(boundary_logits.squeeze(1), target_boundaries)

        # Combined
        total = self.seg_weight * seg_loss + self.boundary_weight * boundary_loss

        return total, seg_loss, boundary_loss

def calculate_iou(pred, target, num_classes):
    """Calculate IoU per class"""
    ious = []
    pred, target = pred.view(-1), target.view(-1)

    for cls in range(num_classes):
        pred_inds = pred == cls
        target_inds = target == cls
```

```

        intersection = (pred_inds & target_inds).sum().item()
        union = (pred_inds | target_inds).sum().item()
        ious.append(intersection / union if union > 0 else float('nan'))

    valid_ious = [iou for iou in ious if not np.isnan(iou)]
    return ious, np.mean(valid_ious) if valid_ious else 0.0

criterion = CMSALoss(config.SEG_WEIGHT, config.BOUNDARY_WEIGHT)

print(f"\nLoss created:")
print(f"    Segmentation: {config.SEG_WEIGHT*100:.0f}%")
print(f"    Boundary: {config.BOUNDARY_WEIGHT*100:.0f}%")

print("\n" + "="*80)
print("SECTION 8 COMPLETE")
print("="*80 + "\n")

# =====
# SECTION 9: TRAINING FUNCTIONS
# =====
print("="*80)
print("SECTION 9: TRAINING FUNCTIONS")
print("="*80)

def train_one_epoch(model, dataloader, criterion, optimizer, device, epoch, scaler, accum):
    """Train one epoch"""
    model.train()
    running_loss = 0.0
    running_seg = 0.0
    running_bnd = 0.0
    all_preds, all_targets = [], []

    optimizer.zero_grad()

    pbar = tqdm(dataloader, desc=f'Epoch {epoch} [Train]')
    for batch_idx, (images, masks, _) in enumerate(pbar):
        images, masks = images.to(device), masks.to(device)

        with autocast(enabled=config.USE_MIXED_PRECISION):
            seg_out, bnd_out, _ = model(images)
            loss, seg_loss, bnd_loss = criterion(seg_out, bnd_out, masks)
            loss = loss / accum

        scaler.scale(loss).backward()

        if (batch_idx + 1) % accum == 0:
            scaler.unscale_(optimizer)
            torch.nn.utils.clip_grad_norm_(model.parameters(), config.GRADIENT_CLIP_NORM)
            scaler.step(optimizer)
            scaler.update()
            optimizer.zero_grad()

        running_loss += loss.item() * images.size(0) * accum
        running_seg += seg_loss.item() * images.size(0)
        running_bnd += bnd_loss.item() * images.size(0)

    preds = torch.argmax(seg_out, dim=1)
    all_preds.append(preds.cpu().numpy())
    all_targets.append(masks.cpu().numpy())

    pbar.set_postfix({
        'loss': f'{loss.item()*accum:.4f}',
        'seg': f'{seg_loss.item():.3f}',
        'bnd': f'{bnd_loss.item():.3f}'
    })
    epoch_loss = running_loss / len(dataloader.dataset)

```

```

epoch_seg = running_seg / len(dataloader.dataset)
epoch_bnd = running_bnd / len(dataloader.dataset)

all_preds = np.concatenate([p.flatten() for p in all_preds])
all_targets = np.concatenate([t.flatten() for t in all_targets])
ious, miou = calculate_iou(torch.from_numpy(all_preds),
                           torch.from_numpy(all_targets),
                           config.NUM_CLASSES)

return epoch_loss, miou, ious, epoch_seg, epoch_bnd

@torch.no_grad()
def validate(model, dataloader, criterion, device, epoch):
    """Validate"""
    model.eval()
    running_loss = 0.0
    all_preds, all_targets = [], []

    pbar = tqdm(dataloader, desc=f'Epoch {epoch} [Val]')
    for images, masks, _ in pbar:
        images, masks = images.to(device), masks.to(device)

        with autocast(enabled=config.USE_MIXED_PRECISION):
            seg_out, bnd_out, _ = model(images)
            loss, _, _ = criterion(seg_out, bnd_out, masks)

        running_loss += loss.item() * images.size(0)
        preds = torch.argmax(seg_out, dim=1)
        all_preds.append(preds.cpu().numpy())
        all_targets.append(masks.cpu().numpy())

    epoch_loss = running_loss / len(dataloader.dataset)
    all_preds = np.concatenate([p.flatten() for p in all_preds])
    all_targets = np.concatenate([t.flatten() for t in all_targets])
    ious, miou = calculate_iou(torch.from_numpy(all_preds),
                               torch.from_numpy(all_targets),
                               config.NUM_CLASSES)

    return epoch_loss, miou, ious

print("Training functions defined!")

print("\n" + "="*80)
print("SECTION 9 COMPLETE")
print("="*80 + "\n")

# =====
# SECTION 10: MAIN TRAINING LOOP
# =====
print("="*80)
print("SECTION 10: MAIN TRAINING")
print("="*80)

def main_training():
    """Main training loop"""

    print("\n" + "="*80)
    print("STARTING CMSA-NET TRAINING (3-CLASS)")
    print("="*80)
    print(f"\nTarget: 82-84% mIoU")
    print(f"Parameters: {total_params/1e6:.2f}M")
    print(f"Classes: {config.CLASS_NAMES}")
    print(f"Novel components:")
    print(f"  1. Cross-Modal Spatial Attention")
    print(f"  2. Hierarchical Multi-Scale Fusion")

```

```

print(f"    3. Boundary-Aware Refinement")

# Datasets with augmentation
train_dataset_aug = WeedsGaloreDataset(
    config.BASE_DIR, config.SPLITS_DIR, 'train',
    train_transform, config.IMG_SIZE
)
val_dataset_clean = WeedsGaloreDataset(
    config.BASE_DIR, config.SPLITS_DIR, 'val',
    None, config.IMG_SIZE
)

# Dataloaders
train_loader = DataLoader(
    train_dataset_aug, batch_size=config.BATCH_SIZE,
    shuffle=True, num_workers=0, pin_memory=True
)
val_loader = DataLoader(
    val_dataset_clean, batch_size=config.BATCH_SIZE,
    shuffle=False, num_workers=0, pin_memory=True
)

print(f"\nDataloaders ready:")
print(f"    Train: {len(train_dataset_aug)} tiles")
print(f"    Val: {len(val_dataset_clean)} tiles")

# Optimizer & scheduler
optimizer = optim.AdamW(model.parameters(), lr=config.LEARNING_RATE,
    weight_decay=config.WEIGHT_DECAY)
scheduler = optim.lr_scheduler.ReduceLROnPlateau(
    optimizer,
    mode='max', # Maximize mIoU
    factor=config.PLATEAU_FACTOR,
    patience=config.PLATEAU_PATIENCE,
    min_lr=config.MIN_LR
)
scaler = GradScaler(enabled=config.USE_MIXED_PRECISION)

# Training loop
best_miou = 0.0
patience = 0
history = []

print("\n" + "="*80)
print("TRAINING STARTED!")
print("="*80)

for epoch in range(1, config.NUM_EPOCHS + 1):
    current_lr = optimizer.param_groups[0]['lr']

    print(f"\n{'='*80}")
    print(f"Epoch {epoch}/{config.NUM_EPOCHS} | LR: {current_lr:.2e}")
    print(f"{'='*80}")

    # Train
    train_loss, train_miou, train_iou, train_seg, train_bnd = train_one_epoch(
        model, train_loader, criterion, optimizer, device, epoch,
        scaler, config.GRADIENT_ACCUMULATION
    )

    # Validate
    val_loss, val_miou, val_iou = validate(
        model, val_loader, criterion, device, epoch
    )

    # Scheduler step
    old_lr = optimizer.param_groups[0]['lr']
    scheduler.step(val_miou)
    new_lr = optimizer.param_groups[0]['lr']

```

```

# Manual verbose output
if new_lr != old_lr:
    print(f"\nReduceLRonPlateau: reducing LR to {new_lr:.2e}")

# Log
history.append({
    'epoch': epoch,
    'train_loss': train_loss,
    'val_loss': val_loss,
    'train_miou': train_miou,
    'val_miou': val_miou,
    'train_seg': train_seg,
    'train_bnd': train_bnd,
    'lr': current_lr
})

print(f"\nResults:")
print(f"    Train Loss: {train_loss:.4f} | Val Loss: {val_loss:.4f}")
print(f"    Train mIoU: {train_miou:.4f} | Val mIoU: {val_miou:.4f}")
print(f"    Loss Components: Seg={train_seg:.4f}, Boundary={train_bnd:.4f}")

print(f"\nPer-class IoU (Validation):")
for name, iou in zip(config.CLASS_NAMES, val_ious):
    if not np.isnan(iou):
        print(f"    {name:12s}: {iou:.4f}")

# Save best
if val_miou > best_miou:
    improvement = val_miou - best_miou
    best_miou = val_miou
    patience = 0

# Save checkpoint
torch.save({
    'epoch': epoch,
    'model_state_dict': model.state_dict(),
    'optimizer_state_dict': optimizer.state_dict(),
    'scheduler_state_dict': scheduler.state_dict(),
    'val_miou': val_miou,
    'val_ious': val_ious,
    'history': history,
    'config': {
        'rgb_filters': config.RGB_FILTERS,
        'nir_filters': config.NIR_FILTERS,
        're_filters': config.RE_FILTERS,
        'decoder_dims': config.DECODER_DIMS,
        'num_classes': config.NUM_CLASSES
    }
}, config.MODEL_SAVE_PATH)

print(f"\nBest model saved! mIoU: {val_miou:.4f} (+{improvement:.4f})")
else:
    patience += 1
    print(f"    Patience: {patience}/{config.EARLY_STOPPING_PATIENCE}")

# Early stopping
if patience >= config.EARLY_STOPPING_PATIENCE:
    print(f"\nEarly stopping at epoch {epoch}")
    break

# Save periodic checkpoint
if epoch % config.SAVE_FREQUENCY == 0:
    checkpoint_path = os.path.join(config.CHECKPOINT_DIR, f'epoch_{epoch}.pth')
    torch.save({
        'epoch': epoch,
        'model_state_dict': model.state_dict(),
        'val_miou': val_miou
    }, checkpoint_path)
    print(f"    Checkpoint saved: epoch_{epoch}.pth")

```



```

        torch.cuda.empty_cache()

    # Save training history
    history_df = pd.DataFrame(history)
    history_df.to_csv(config.LOG_PATH, index=False)
    print(f"\nTraining history saved: {config.LOG_PATH}")

    print("\n" + "="*80)
    print("TRAINING COMPLETE!")
    print("="*80)
    print(f"\nFINAL RESULTS:")
    print(f"    Best validation mIoU: {best_miou:.4f}")
    print(f"    Target: 82-84%")
    print(f"    Total parameters: {total_params/1e6:.2f}M")
    print(f"\nBest model: {config.MODEL_SAVE_PATH}")
    print(f"Checkpoints: {config.CHECKPOINT_DIR}")
    print(f"Training log: {config.LOG_PATH}")

    return history, best_miou

# RUN TRAINING
print("Starting training pipeline...")
history, best_miou = main_training()

print("\n" + "="*80)
print("ALL SECTIONS COMPLETE")
print("="*80)
print(f"\nCMSA-NET TRAINING FINISHED!")
print(f"    Best mIoU: {best_miou:.4f}")
print(f"    Model: {config.MODEL_SAVE_PATH}")

```